

My Project

Generated by Doxygen 1.13.2

Chapter 1

Aprašymas

Programa, skirta apdoroti studentų duomenis. Programos versijose analizuojamas bei tobulinamas programos našumas, efektyvumas bei paruošimas vartotojų.

Yra sukurta programos dokumentacija - `dokumentacija.pdf`.

1.1 Naudojimo instrukcijos

Katalogai:

- `analysis` - programos veikimo analizės rezultatai, naudojami tam tikrose programos versijose
- `docs` - doxygen failai
- `files` - programoje vykdymo metu naudojami `.txt` failai (įtrauktas į `.gitignore` ir atsiras tik paleidus programą)
- `include` - antraščių `.h` failai
- `src` - `.cpp` failai

Kad paleisti programą, turite atlikti šiuos veiksmus:

1. Įeikite į norimos versijos katalogą

2. Paleiskite `run.bat` failą

`run.bat` failas atliks šiuos veiksmus:

- Sukurs `build` katalogą
- Paleis CMake, kad sugeneruotų `Makefile`
- Sukompiliuos projektą naudodamas `make` komandą
- Paleis sukompiliuotą programą terminale

Reikalavimai

- Operacinė sistema: Windows 10 x64-bit arba naujesnė versija
- Įdiegta CMake (3.25 arba naujesnė versija)
- Kompiliatorius: g++ (su C++11 arba naujesne versija)

1.1.1 Duomenų įvestis

Tipas	Aprašymas
Rankinis	Vartotojas suveda duomenis ranka tiesiai į konsolę.
Automatinis	Naudojamas vartotojui norint sugeneruoti atsitiktinius duomenis bei testavimui, naudojamas <code>istringstream</code> .
Iš failo	Skaitomi duomenys iš failo naudojant <code>ifstream</code> (v1.2 teste numatytas vienas dydis - 3, kitur galima rinktis iš 5 dydžių).

1.1.2 Duomenų išvestis

Tipas	Aprašymas
Į ekraną	Duomenys atvaizduojami konsolėje per <code>cout <<</code>
Į failą	Duomenys saugomi faile per <code>ostream</code>

1.1.3 Kompiliavimas programavimo aplinkoje

Tipas	Aprašymas
Programa	<code>cd build && .\Objektinis.exe</code> (nukreipia į MENU)
Catch2 testai	<code>cd build && .\tests.exe</code>

1.1.4 MENU

Paleidus programą, vartotojas pateks į meniu, kuriame galės pasirinkti ar nori vykdyti programą ar tyrimus. Žemiau pateikiami atitinkami MENU, kurie atsiras po pasirinkimo.

1.1.5 Programos MENU

Parinktis	Aprašymas
1	Įvesti viską rankiniu būdu
2	Generuoti atsitiktinius pažymius (vardus įvesti ranka)
3	Generuoti atsitiktinius vardus ir pažymius
4	Skaityti iš failo
5	Užbaigti programą

1.1.6 Tyrimų MENU

Parinktis	Aprašymas
1	Failų generavimas
2	Programos veikimo laikas
3	Rule of Five testavimas
4	Nuosavo vektoriaus testavimas
5	Vector ir <code>std::vector</code> paskirstymo testavimas
0	Užbaigti programą

1.2 Versija v3.0

Šioje versijoje implementuotas nuosavas vektorius `Vector`, kuris padengia daugiau nei 80% `std::vector` funkcionalumo. Vektorius aprašytas `include/ownVector.h` faile.

1.2.1 Vector<T> Klasės Aprašymas

Šiame skyriuje aprašomi `Vector<T>` klasės metodai, kurie suteikia dinaminio masyvo funkcionalumą, panašų į `std::vector`.

1.2.1.1 Konstruktoriai

Funkcija	Aprašymas
<code>Vector()</code>	Numatytasis konstruktorius. Sukuria tuščią vektorių.
<code>Vector(size_t n)</code>	Sukuria vektorių su <code>n</code> elementų.
<code>Vector(size_t n, const T& value)</code>	Sukuria vektorių su <code>n</code> elementų, inicijuotų nurodyta <code>value</code> .
<code>Vector(std::initializer_list<T> init)</code>	Sukuria vektorių iš inicializavimo sąrašo.

1.2.1.2 Rule of Five Metodai

Funkcija	Aprašymas
<code>Vector(const Vector<T>& other)</code>	Kopijavimo konstruktorius. Sukuria vektoriaus kopiją.
<code>Vector(Vector<T>&& other) noexcept</code>	Perkėlimo konstruktorius. Perkėlia kitą vektorių.
<code>~Vector()</code>	Destruktorius. Atlaisvina užimtą atmintį.
<code>Vector<T>& operator=(const Vector<T>& other)</code>	Kopijavimo priskyrimo operatorius. Kopijuoja kito vektoriaus turinį.
<code>Vector<T>& operator=(Vector<T>&& other) noexcept</code>	Perkėlimo priskyrimo operatorius. Perkėlia kito vektoriaus turinį.

1.2.1.3 Elementų Prieiga

Funkcija	Aprašymas
<code>operator[](size_t index)</code>	Tiesioginė prieiga prie elemento pagal indeksą (nera saugus).
<code>const operator[](size_t index) const</code>	Konstantiška versija.
<code>back()</code>	Grąžina paskutinį vektoriaus elementą.
<code>const back() const</code>	Konstantiška versija.
<code>front()</code>	Grąžina pirmą vektoriaus elementą.
<code>const front() const</code>	Konstantiška versija.
<code>data()</code>	Grąžina rodyklę į pirmojo elemento masyvą.
<code>const data() const</code>	Konstantiška versija.
<code>at(size_t index)</code>	Saugus pasiekimas prie elemento pagal indeksą; meta <code>std::out_of_range</code> jei indeksas už ribų.
<code>const at(size_t index) const</code>	Konstantiška versija.

1.2.1.4 Iteratoriai

Funkcija	Aprašymas
<code>begin()</code>	Grąžina rodyklę į pirmą elementą .
<code>const begin() const</code>	Konstantiška versija.
<code>end()</code>	Grąžina rodyklę į elementą už paskutinio .
<code>const end() const</code>	Konstantiška versija.

1.2.1.5 Dydis ir Talpa

Funkcija	Aprašymas
<code>size() const</code>	Grąžina elementų kiekį (dydį) vektoriuje.
<code>capacity() const</code>	Grąžina rezervuotą atminties kiekį .
<code>empty() const</code>	Patikrina, ar vektorius yra tuščias .

1.2.1.6 Atminties Valdymas

Funkcija	Aprašymas
<code>reserve(size_t new_cap)</code>	Rezervuoja nurodytą atminties kiekį (<code>new_cap</code>).
<code>shrink_to_fit()</code>	Sumažina talpą iki dabartinio elementų skaičiaus (dydžio).
<code>reallocate(size_t new_cap)</code>	(Privatus) Perplanuoja atmintį. Naudojamas vidiniams perskirstymams.
<code>getReallocationCount() const</code>	Grąžina atminties perskirstymų skaičių .

1.2.1.7 Modifikavimo Funkcijos

Funkcija	Aprašymas
<code>push_back(const T& value)</code>	Prideda elementą į vektoriaus galą (kopija).
<code>push_back(T&& value)</code>	Prideda elementą į vektoriaus galą (perkėlimas).
<code>emplace_back(Args&&... args)</code>	Konstruoja elementą tiesiogiai vektoriaus gale. Tai efektyviau, nes išvengiama kopijavimo/perkėlimo.
<code>pop_back()</code>	Pašalina paskutinį elementą.
<code>clear()</code>	Išvalo vektorių (dydis tampa 0).
<code>resize(size_t new_size)</code>	Keičia vektoriaus dydį. Nauji elementai neinicijuojami.
<code>resize(size_t new_size, const T& value)</code>	Keičia vektoriaus dydį ir inicijuoja naujus elementus nurodyta <code>value</code> .
<code>insert(size_t index, const T& value)</code>	Įterpia elementą nurodytoje pozicijoje.
<code>erase(size_t index)</code>	Pašalina elementą nurodytoje pozicijoje.
<code>swap(Vector<T>& other)</code>	Sukeičia dviejų vektorių turinį.
<code>assign(size_t count, const T& value)</code>	Priskiria <code>count</code> elementų su ta pačia <code>value</code> .

1.2.1.8 Palyginimo Operatoriai

Funkcija	Aprašymas
<code>operator==(const Vector<T>& other) const</code>	Patikrina, ar du vektoriai yra lygūs .

Funkcija	Aprašymas
<code>operator!=(const Vector<T>& other) const</code>	Patikrina, ar du vektoriai yra nelygūs .
<code>operator<(const Vector<T>& other) const</code>	Patikrina, ar dabartinis vektorius yra mažesnis už kitą.
<code>operator>(const Vector<T>& other) const</code>	Patikrina, ar dabartinis vektorius yra didesnis už kitą.

1.2.1.9 Paieškos Funkcijos

Funkcija	Aprašymas
<code>contains(const T& value) const</code>	Patikrina, ar vektoriuje yra nurodyta <code>value</code> .
<code>index_of(const T& value) const</code>	Grąžina pirmosios <code>value</code> pasikartojimo indeksą, arba <code>-1</code> jei nerasta.

1.2.1.10 Kitos Naudingos Funkcijos

Funkcija	Aprašymas
<code>slice(size_t start, size_t end) const</code>	Grąžina naują vektorių, sudarytą iš elementų nuo <code>start</code> iki <code>end-1</code> .
<code>sort()</code>	Rūšiuoja vektoriaus elementus.
<code>unique()</code>	Pašalina pasikartojančius elementus iš surūšiuoto vektoriaus.
<code>map(std::function<T(const T&)> func) const</code>	Grąžina naują vektorių, pritaikytą funkciją kiekvienam elementui.
<code>reverse()</code>	Apverčia vektoriaus elementų tvarką.
<code>remove(const T& value)</code>	Pašalina pirmą pasikartojančią nurodytą reikšmę.

1.2.2 Pradinis Vector testas

Atliktas pradinis Vector testas lyginant su `std::vector`. Patikrintas bazinis funkcionalumas. Testas aprašytas `src/functions.cpp` faile -> `void testOwnVector()`. Jį galima įvykdyti pasirinkus Menu -> testavimas -> nuosavo vektoriaus testavimas.

Visi pradiniai testai praėjo sėkmingai.

1.2.3 UNIT testai

Atlikti catch2 unit testai pilnam nuosavo vektoriaus klasės funkcionalumui ištirti. Aprašyti `src/catchTest.cpp` faile.

Visi catch testai praėjo sėkmingai.

1.2.4 Efektyvumo testai

Atlikti efektyvumo testai lyginant `std::vector` ir nuosavą `Vector`, tuščius vektorius užpildant: 10000, 100000, 1000000, 10000000 ir 100000000 int elementų naudojant `push_back()` funkciją. Testas aprašytas `src/functions.cpp` faile -> `benchmarkPushBack`. Testavimui naudota `std::chrono::high_resolution_clock` biblioteka.

Elementų skaičius	std::vector (s)	Own Vector (s)	std::vector reallocations	Own Vector reallocations
10 000	0.00047	0.00015	15	15
100 000	0.00277	0.00120	18	18
1 000 000	0.02323	0.00635	21	21
10 000 000	0.18471	0.07862	25	25
100 000 000	1.86080	0.68589	28	28

Išvados:

- Nuosavas `Vector` konteineris parodė geresnį našumą už `std::vector`, ypač užpildant didesnius kiekius elementų.
- Perskirstymai (reallocations) nuosavam `Vector` vyksta maždaug 15-28 kartus priklausomai nuo duomenų kiekio, kas atitinka dvigubino strategiją atminties valdyme ir yra tokie aptys kaip ir `std::vector`.

1.3 Programos versijos

Kiekviena versija išsamiai aprašyta jos `README.md` faile. Programos v1.0 versijos yra atskiroje `opp` repozitorijoje.

1.3.1 v.pradinė

C++ programa, skirta studentų pažymiams apskaičiuoti, naudojant vidurkio ir medianos metodus. Vartotojai įveda studentų vardus, namų darbų pažymius ir egzaminų rezultatus, o programa pateikia suformatuotus rezultatus. `Makefile` automatizuoja kompiliavimą, o `gitignore` pašalina nereikalingus failus iš „Git“ sekimo.

1.3.2 v0.1

Optimizuoti failai ir pataisytos galutinio pažymio apskaičiavimo funkcijos. Įgyvendinti du skirtingi vykdomieji failai: vienas su vektorių masyvais, kitas – su dinaminiais C masyvais pažymiams. Pridėta `menu` funkcija, leidžianti atsitiktinai generuoti pasirinktus duomenis.

1.3.3 v0.2

Prie `menu` pridėta failų skaitymo parinktis, rūšiavimo parinktys (pagal vardą, pavardę ir galutinį pažymį) ir parinktis pasirinkti, ar spausdinti rezultatą į monitorių, ar į failą. Taip pat pridėti `TimeMeasurement` failai, skirti sekti laiką ir apskaičiuoti vidutinius testavimo laikus.

1.3.4 v0.3

Pridėtas išimčių tvarkymas funkcijoms `readFromFile`, `output` ir `sortStudents`. Sukurti aplankai ir atnaujintas `makefile`.

1.3.5 v0.4

Pridėta failų generavimo funkcija su pasirinktu dydžiu, studentų grupavimas į dvi grupes pagal pasirinktą galutinio pažymio tipą ir dvi laiko matavimo testavimo funkcijos. Pakeista `timeMeasurement` funkcija, kad testų rezultatai būtų spausdinami į failą, o ne į konsolę. Atnaujintas `README`, įtraukiant laiko matavimo testų rezultatus. Optimizuotos kelios funkcijos, siekiant geresnio našumo.

1.3.6 v1.0prad

Pridėta nauja laiko testavimo funkcija ir versija su 3 skirtingais konteneriais: `vector`, `deque` ir `list`. Atliktas laiko tyrimas, o rezultatai pateikti faile `README`.

1.3.7 v1.0

Pridėti testavimai su trimis skirtingomis strategijomis studentų rūšiavimo į grupes funkcijai. Optimizuotas geriausias rūšiavimas (`vector` kontenerio su pirma strategija). Atliktas spartos bei atminties naudojimo tyrimas, o rezultatai pateikti faile `README`. Pridėtas programos diegimo bei paleidimo `CMake` failas bei `README` aprašyta įdiegimo instrukcija.

1.3.8 v1.1

`Vector` kontenerio versijoje duomenų struktūra pakeista iš `struct` į `class`. Atlikti programos spartos bei SSD naudojimo testai lyginant `struct` ir `class` su skirtingomis optimizavimo vėliavėlėmis: `O1`, `O2` ir `O3`. Tyrimo rezultatai aprašyti `README` faile.

1.3.9 v1.2

Pašalinti `deque` bei `list` versijų katalogai. Pritaikyta Rule of Five ir persidengimo operatoriai. Atliktas testas patikrinantis `Student` klasės metodų veikimą, testui naudojami papildomi `.txt` failai kurie susikuria automatiškai.

1.3.10 v1.5

Sukurta nauja bazinė klasė `Zmogus`. `Student` klasė paversta į `derived`. Patikrinta, ar nauji metodai praeina testus iš versijos 1.2 Kad įsitikint, jog klasė `Zmogus` yra abstrakti, reikia atkomentuoti funkciją `void testZmogaus←Class()` failuose `functions.cpp` bei `functions.h`.

1.3.11 v2.0

Sukurta klasė aprašanti dokumentacija, HTML ir TEX formatais, su sukompiliuotu PDF failu. Atlikti Catch2 testai faile `src/catchTest.cpp`.

1.3.12 v3.0

Implementuotas nuosavas `Vector` padengiantis daugiau nei 80% `std::vector` funkcionalumo. Atlikti efektyvumo bei funkcionalumo testavimai, programa perrašyta ant nuosavo `Vector`.

